UNIVERSITY OF MINNESOTA
Driven to Discover®

Runtime Gameplay Foundation Systems

CSCI 5980: Game Engine Architecture

Evan Suma Rosenberg | CSCI 5980 | Spring 2026

The Gameplay Foundation System

- Most engines provide a suite of runtime components for building a game's rules, objectives, and dynamic world elements.
- We refer to them collectively as the **gameplay foundation system**.
- These systems lie just beneath the line between engine and game.
- The line between engine and game is best visualized as a gradient.
- Differences between engines are most acute in their gameplay components.

Major Subsystems

- Runtime game object model
- Level management and streaming
- Real-time object model updating
- Messaging and event handling
- Scripting
- Objectives and game flow management

Runtime Object Model Features

- Spawning and destroying objects dynamically
- Linkage to low-level engine systems
  (rendering, animation, audio, collision, physics)
- Real-time simulation of object behaviors
- Ability to define new game object types
  (ideally data-driven)
- Unique object ids
  (human-readable name, integer, or hashed string)

Runtime Object Model Features

- Game object queries
  (by id, by type, by criteria)
- Game object references
  (pointers, handles, smart pointers)
- Finite state machine support
- Network replication for multiplayer
- Saving and loading / object persistence

Runtime Object Model Architectures

Object Model Architectures

- The runtime object model is the in-game manifestation of the tool-side abstract model
- This is by far the largest component of any gameplay foundation system
- Designs vary, but most engines follow one of two basic styles:
- Object-centric: each object is a class instance (or small collection of class instances)
  - Attributes and behaviors are encapsulated within the class(es)
- Property-centric: each object is just a unique id
  - Properties are distributed across data tables, keyed by object id
  - Behavior is implicitly defined by the collection of properties

Object-Centric Architectures

A Simple Model in C: Hydro Thunder

- Game object models needn't be in an OOP language
- Hydro Thunder (Midway Home Entertainment) was written entirely in C
- A few object types:
  - boats, boost icons, ambient animated objects, water surface, ramps, waterfalls, particle effects, race track sectors, static geometry, HUD elements

Hydro Thunder, 2001

WorldOb_t Struct

- Dynamic objects represented by a general-purpose `WorldOb_t` struct
- The struct contained a `void*` "user data" pointer plus pointers to "update" and "draw" functions
- These pointers acted like virtual functions, providing rudimentary inheritance and polymorphism

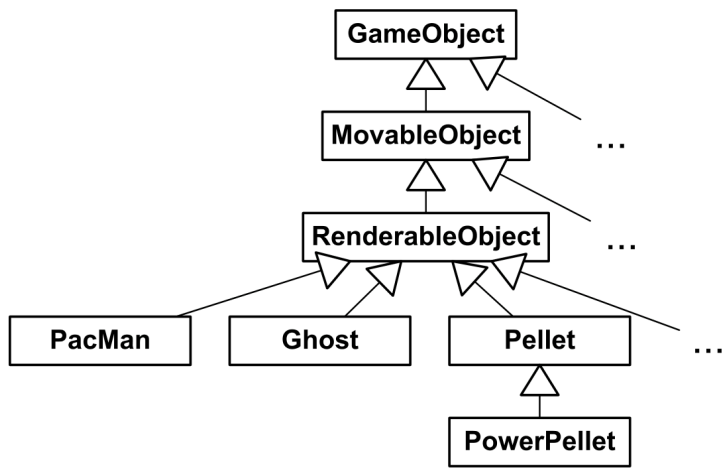
```
struct WorldOb_s
{
    Orient_t m_transform;           /* position/rotation */
    Mesh3d* m_pMesh;                /* 3D mesh */
    /* ... */
    void*    m_pUserData;           /* custom state */

      void          (*m_pUpdate)(); /* polymorphic update */
      void          (*m_pDraw)();   /* polymorphic draw */
};
typedef struct WorldOb_s WorldOb_t;
```

Monolithic Class Hierarchies

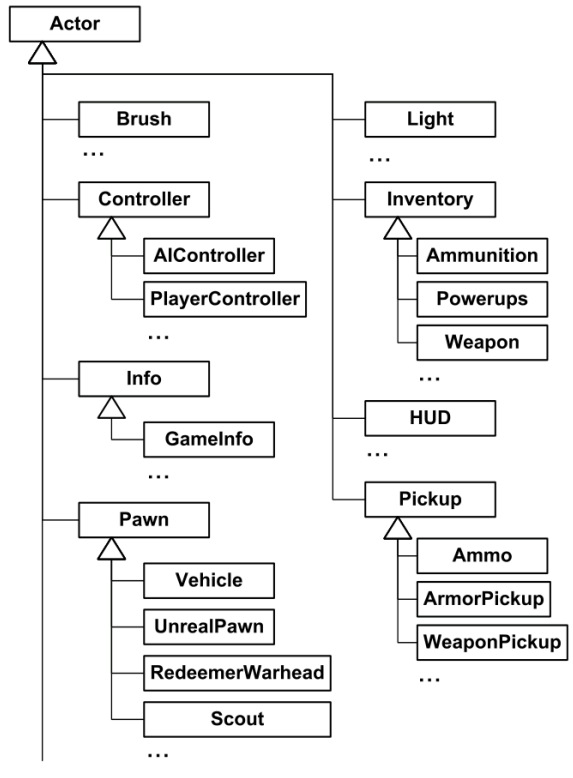
- Game programmers naturally classify object types taxonomically
- A class hierarchy is the most intuitive way to represent interrelated game object types
- Most commercial game engines use a class-hierarchy-based technique
- Hierarchies usually start small and simple, but tend to deepen and widen over time
- A **monolithic class hierarchy** arises when virtually all classes inherit from one common base class
- Unreal Engine's game object model is a classic example

A Pac-Man Class Hierarchy

- Common, generic functionality at the root
- Increasingly specific functionality toward the leaves



Excerpt from Unreal's Hierarchy



Problems with Deep, Wide Hierarchies

- Hard to **understand, maintain, and modify** classes deep in the hierarchy
  - Need to understand all parent classes too
- Cannot describe **multidimensional taxonomies**
  - A hierarchy classifies along one axis only
- **Multiple inheritance** has practical problems (deadly diamond)
- The **bubble-up effect**: features migrate to root over time

The Vehicle Hierarchy Problem

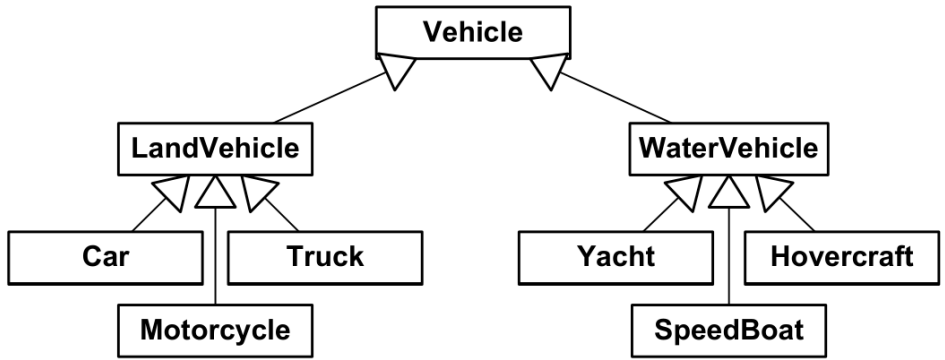
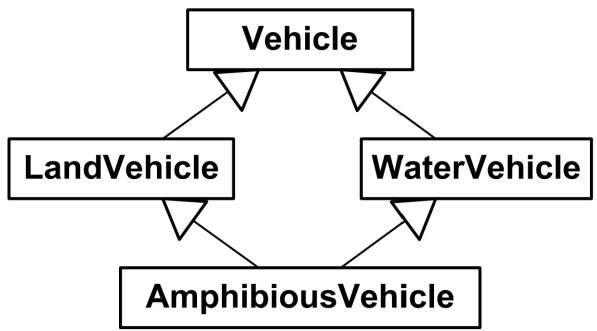


Figure 16.4. A seemingly logical class hierarchy describing various kinds of vehicles.

- What happens when designers want an amphibious vehicle?
- Doesn’t fit the existing taxonomy
- Programmers may panic or "hack" the hierarchy

Multiple Inheritance: The Deadly Diamond

- One solution: use C++ multiple inheritance for amphibious vehicle
- But MI in C++ has practical problems
- Can lead to multiple copies of base class members
- Known as the "deadly diamond" or "diamond of death"
- Difficulties usually outweigh the benefits
- Most game studios prohibit or severely limit multiple inheritance



A diamond-shaped class hierarchy

Mix-In Classes

- A limited form of MI: any number of parents, but only one grandparent
- A class may inherit from one class in the main hierarchy, plus any number of **mix-in classes** (stand-alone classes with no base class)
- Common functionality factored into mix-in classes
- "Spot-patched" into the main hierarchy where needed
- Usually better to compose or aggregate such classes than to inherit from them

Mix-In Classes

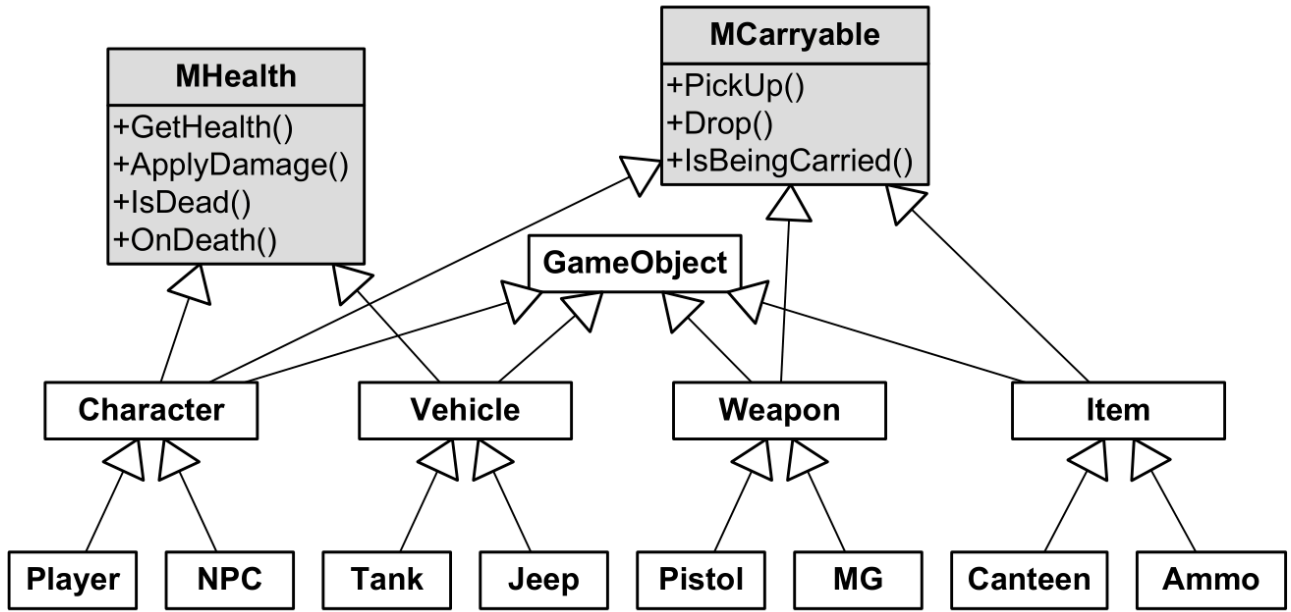


Figure 16.6. A class hierarchy with mix-in classes. The MHealth mix-in class adds the notion of health and the ability to be killed to any class that inherits it. The MCarryable mix-in class allows an object that inherits it to be carried by a Character.

The Bubble-Up Effect

- Root classes start simple, exposing minimal feature sets
- Desire to share code between unrelated classes makes features "bubble up"
- Example: only wooden crates float, then characters float, then bits of paper, vehicles...
- "Floating" was not an original classification criterion
- Programmers move flotation code into a common base class
- Eventually flotation is a feature of the root object
- The Unreal Actor class is a classic example
  - Contains rendering, animation, physics, audio, network replication, creation/destruction, iteration, broadcasting

Composition over Inheritance

- Most prevalent cause of monolithic hierarchies: over-use of "is-a" relationships
- Example: deriving `Window` from `Rectangle` because windows are "always rectangular"
- A window is not a rectangle: it has a rectangle
- Better solution: embed an instance of `Rectangle` inside `Window`
- The "has-a" relationship is composition (class A owns an instance of class B)
- Aggregation: A links to B without managing its lifetime

Limitations of Inheritance-Only

- Limits design choices for new game objects
- A physically simulated object must inherit from `PhysicalObject` even if it doesn't need animation
- Difficult to extend functionality of existing classes
- Adding morph target animation alongside skeletal animation forces refactoring or multiple inheritance

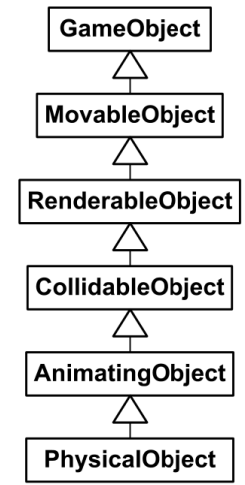


Figure 16.7. A hypothetical game object class hierarchy using only inheritance to associate the classes.

Refactoring with Components

- Isolate features into independent classes called components or service objects
- Each component provides a single, well-defined service
- Select only the features needed for each game object type
- Each feature can be maintained, extended, or refactored without affecting others
- Easier to understand and test (decoupled from one another)
- Some components correspond directly to engine subsystems

Refactoring with Components

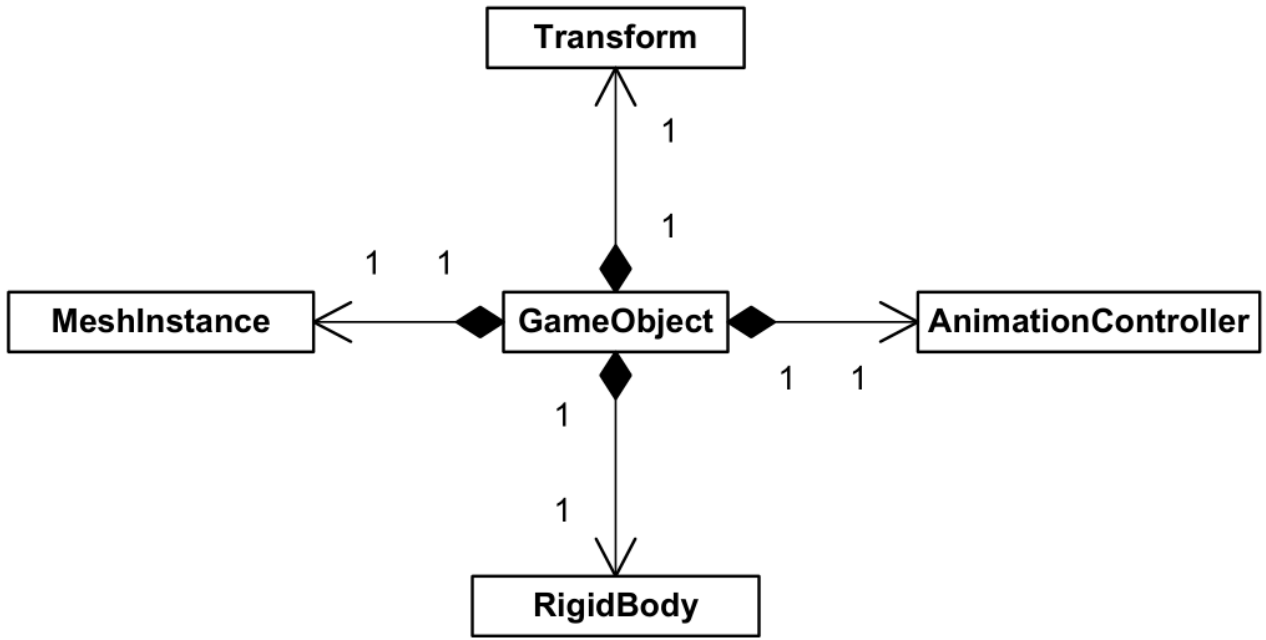


Figure 16.8. Our hypothetical game object class hierarchy, refactored to favor class composition over inheritance.

Component Creation and Ownership

- The "hub" class typically owns its components and manages their lifetimes
- One approach: provide root `GameObject` with pointers to possible components
- Each derived class's constructor creates whatever components it needs
- Default `GameObject` destructor cleans up all components automatically
- The hierarchy of `GameObject`-derived classes serves as the primary taxonomy
- Component classes serve as optional add-on features

GameObject with Components

```
class GameObject
{
protected:
    // My transform (position, rotation, scale).
    Transform            m_transform;

    // Standard components:
    MeshInstance*        m_pMeshInst;
    AnimationController* m_pAnimController;
    RigidBody*           m_pRigidBody;

public:
    GameObject()
    {
        // Assume no components by default.
        // Derived classes will override.
        m_pMeshInst = nullptr;
        m_pAnimController = nullptr;
        m_pRigidBody = nullptr;
    }
```

GameObject Destructor

```
    ~GameObject()
    {
        // Automatically delete any components created by
        // derived classes. (Deleting null pointers OK.)
        delete m_pMeshInst;
        delete m_pAnimController;
        delete m_pRigidBody;
    }

    // ...
};
```

Vehicle Subclass

```
class Vehicle : public GameObject
{
protected:
    // Add some more components specific to Vehicles ...
    Chassis* m_pChassis;
    Engine* m_pEngine;
    //  ...

public:
    Vehicle()
    {
        // Construct standard GameObject components.
        m_pMeshInst = new MeshInstance;
        m_pRigidBody = new RigidBody;

        // NOTE: We'll assume the animation controller must be provided with a reference to the mesh
        // instance so that it can provide it with a  matrix palette.
        m_pAnimController = new AnimationController(*m_pMeshInst);

        // Construct vehicle-specific components.
        m_pChassis = new Chassis(*this, *m_pAnimController);
        m_pEngine = new Engine(*this);
    }
```

Vehicle Subclass (continued)

```
    ~Vehicle()
    {
        // Only need to destroy vehicle-specific
        // components, as GameObject cleans up the
        // standard components for us.
        delete m_pChassis;
        delete m_pEngine;
    }
};
```

Generic Components

- Alternative: provide root game object with a generic linked list of components
- Components derive from a common base class
- Allows polymorphic operations (asking type, passing events to each)
- Root class can be largely oblivious to component types
  - Permits new components to be created without modifying the root class
- Allows arbitrary numbers of instances of each type
- Trickier to implement: code must be totally generic
- Neither hard-coded nor generic design is clearly superior

Generic Components

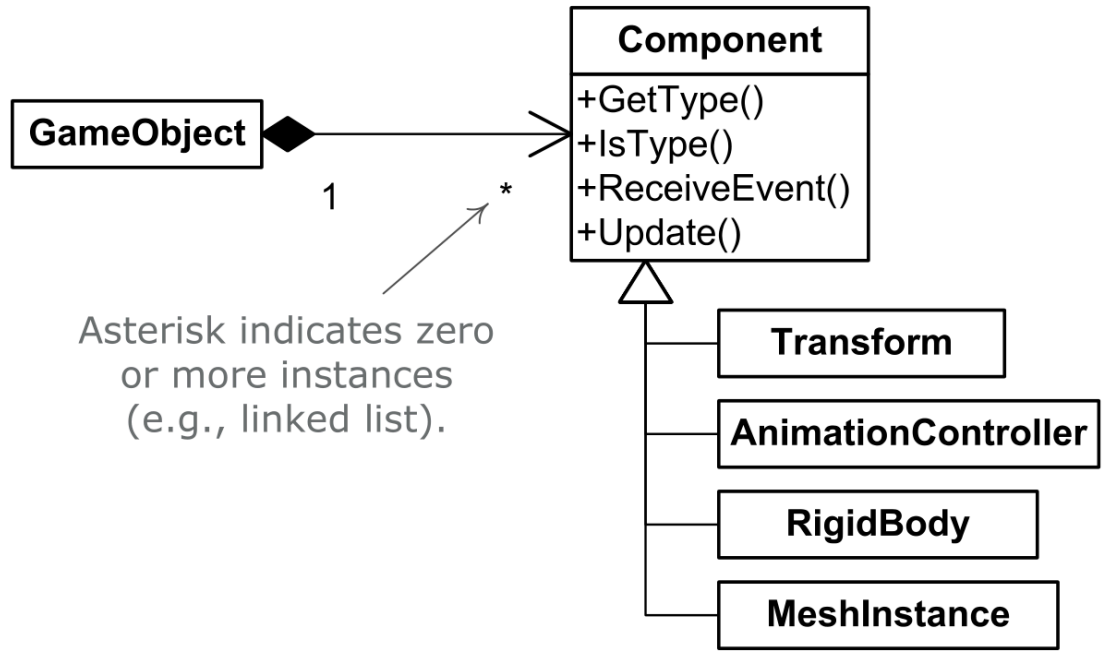


Figure 16.9. A linked list of components can provide flexibility by allowing the hub game object to be unaware of the details of any particular component.

Pure Component Models

- Take componentization to the extreme: move all functionality into components
- The root `GameObject` becomes a behavior-less container
  - Why not eliminate it entirely?
- Give each component a copy of the unique id; link components by shared id
- Given a way to look up components by id, no "hub" class is needed
- Trade-offs:
  - Need a way to define concrete object types (factory pattern, data-driven)
  - Inter-component communication is harder
  - Sending messages between objects is more difficult

Pure Component Models

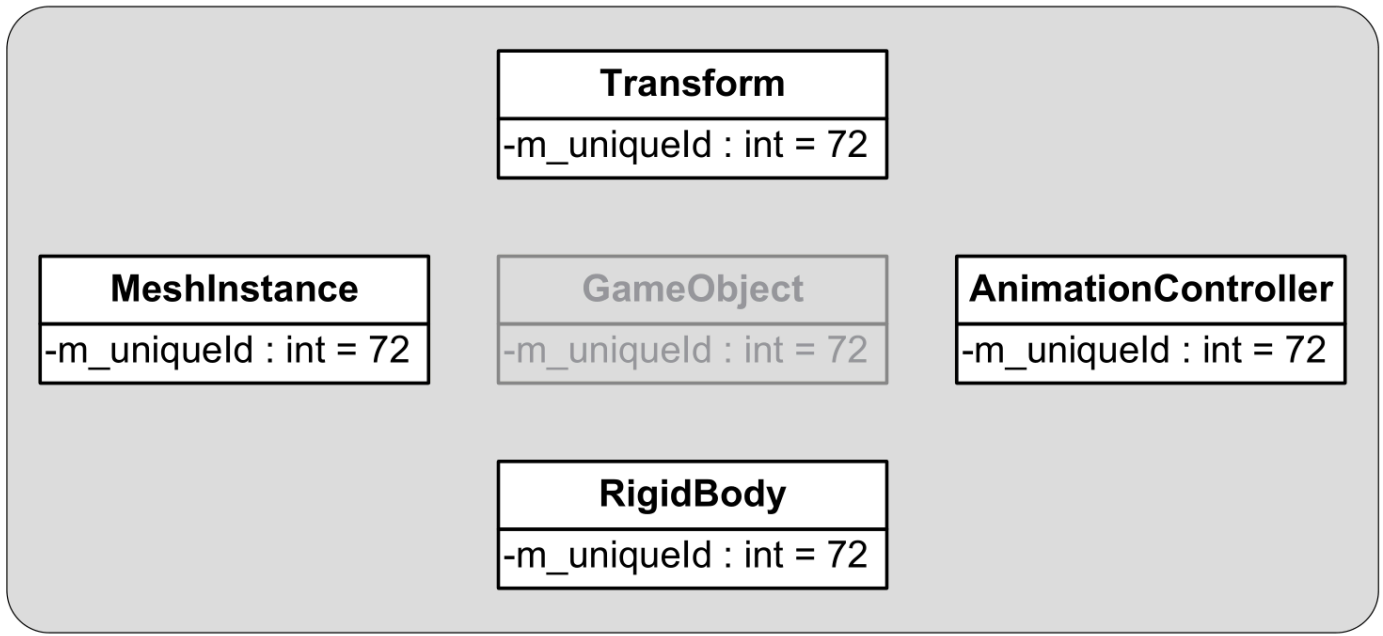


Figure 16.10. In a pure component model, a logical game object is comprised of many components, but the components are linked together only indirectly, by sharing a unique id.

Property-Centric Architectures

Property-Centric View

- Object-centric: think of objects that contain attributes and behaviors
- Property-centric: think primarily in terms of attributes
- Build a table for each property containing values keyed by object id
- More akin to a relational database than an object model
- Each attribute is a table; the unique id is the primary key
- Used successfully on Deus Ex 2 and the Thief series

Object-Centric vs Property-Centric

Object-centric:

- Object1: Position = (0, 3, 15), Orientation = (0, 43, 0)
- Object2: Position = (-12, 0, 8), Health = 15
- Object3: Orientation = (0, -87, 10)

Property-centric:

- Position: Object1 = (0, 3, 15), Object2 = (-12, 0, 8)
- Orientation: Object1 = (0, 43, 0), Object3 = (0, -87, 10)
- Health: Object2 = 15

Behavior via Property Classes

- Each property can be implemented as a class
- Properties can be simple (a `bool` or `float`) or complex (a renderable mesh, an AI brain)
- Each class provides behaviors via its hard-coded methods
- Overall behavior is the aggregation of all property behaviors
- Example: a `Health` property responds to attacks, decrements health
- Properties can communicate with sibling properties
  - `Health` can ask `AnimatedSkeleton` to play a hit reaction
  - On death, `Health` can tell `RigidBodyDynamics` to start a rag doll

Behavior via Script

- Store property values as raw data in database-like tables
- Use script code to implement behaviors
- A `ScriptId` property specifies the block of script code that manages the object
- Script can also let an object respond to events
- Some engines provide hard-coded property classes, plus the ability to define new property types in script
  - This approach was used on Dungeon Siege

Entity-Component Systems

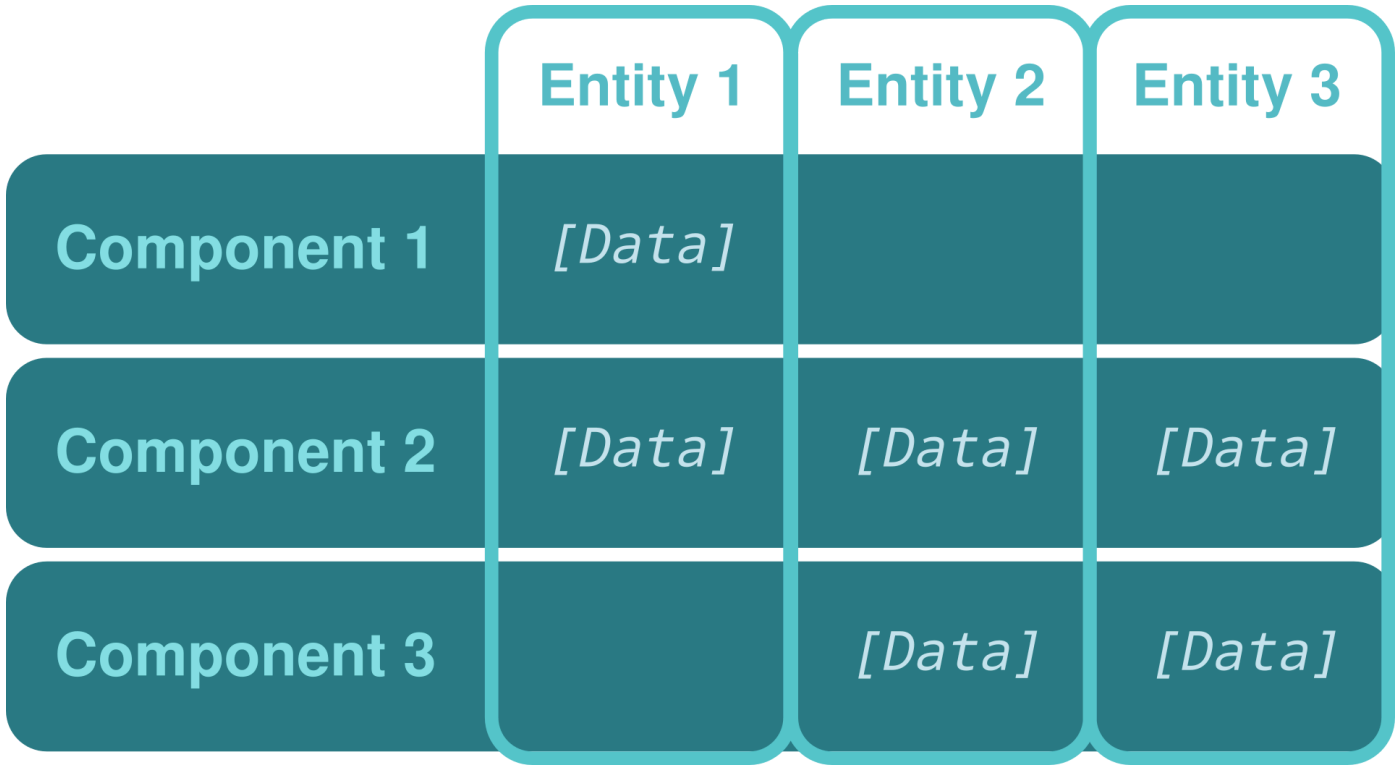


Image Credit: https://en.wikipedia.org/wiki/Entity_component_system

Properties versus Components

- Many authors use "component" to refer to what is called a "property object" here
- Property objects are very closely related to components
- Both designs: a single logical object made up of multiple subobjects
- Main distinction: the roles of the subobjects
  - Property-centric: each subobject defines an attribute (health, inventory, etc.)
  - Object-centric: subobjects represent linkages to engine subsystems
- In practice, the distinction is often virtually irrelevant

Pros and Cons of Property-Centric

- More memory-efficient: only store attribute data actually in use
- Easier to construct in a data-driven manner
- More cache-friendly: data of the same type is contiguous in memory
- This is the struct of arrays technique
  - Versus the more traditional array of structs approach
- On the PS3, a single cache miss equals thousands of CPU instructions
- Storing data contiguously reduces or eliminates cache misses

ECS Engine Example: Bevy

Querying and Updating Game Objects in Real Time

Game Object Queries

- Finding game objects at runtime
- Simplest: by unique id
- Other examples:
  - Find all enemies with line of sight to the player
  - Iterate over all objects of a certain type
  - Find all destructible objects with more than 80% health
  - Transmit damage to objects within an explosion’s radius
  - Iterate objects in a bullet’s path, nearest-to-farthest

Specialized Query Data Structures

- A general-purpose database with arbitrary fast queries is impractical
- Teams identify likely queries and build specialized structures
- By unique id: hash table or binary search tree
- By criterion: presort into linked lists (e.g., by type, by player proximity)
- In a projectile's path / line of sight: leverage collision system's ray and shape casts
- Within a region or radius: spatial hash, grid, quadtree, octree, kd-tree

The Game Loop and Object Updates

- Most low-level subsystems require periodic updating
- The game object system is no exception
- Updating is done via the game loop
- Game object updating: convert $S_i(t - \Delta t)$ into $S_i(t)$
- Once all object states are updated, t becomes the new $t - \Delta t$
- The clock that drives object updates is allowed to diverge from real time
  - Permits pause, slow-mo, fast-forward, reverse, debugging

A Simple Approach

- The simplest approach: iterate the collection and call a virtual `Update()` on each
- Done once per frame
- Each class provides a custom `Update()` implementation
- The time delta from the previous frame is passed in
- We assume a monolithic object hierarchy, but ideas extend to other designs

```
virtual void Update(float dt);
```

Maintaining the Active Object Collection

- Often a singleton manager, e.g. `GameWorld` or `GameObjectManager`
- Generally needs to be dynamic (objects spawn and destroy)
- Often a linked list of pointers, smart pointers, or handles
- Engines that disallow dynamic spawning can use a statically sized array
- Most engines use more complex data structures, but visualize as a linked list

Responsibilities of Update()

- Determine state $S_i(t)$ from $S_i(t - \Delta t)$
- May involve rigid body dynamics, sampling animations, reacting to events
- Most objects interact with engine subsystems
  - Animate, render, particle effects, audio, collision
- Naive idea: have each object update its subsystems directly

```
m_pAnimationComponent->Update(dt);
m_pCollisionComponent->Update(dt);
m_pPhysicsComponent->Update(dt);
m_pAudioComponent->Update(dt);
m_pRenderingComponent->draw();
```

- This is NOT a good idea

A Hypothetical Tank Update

```
virtual void Tank::Update(float dt)
{
    // Update the state of the tank itself.
    MoveTank(dt);
    DeflectTurret(dt);
    FireIfNecessary();

    // Now update low-level engine subsystems on behalf
    // of this tank. (NOT a good idea ... see below!)
    m_pAnimationComponent->Update(dt);
    m_pCollisionComponent->Update(dt);
    m_pPhysicsComponent->Update(dt);
    m_pAudioComponent->Update(dt);
    m_pRenderingComponent->draw();
}
```

A Naive Game Loop

```
while (true)
{
    PollJoypad();

    float dt = g_gameClock.CalculateDeltaTime();

    for (each gameObject)
    {
        // This hypothetical Update() function updates
        // all engine subsystems!
        gameObject.Update(dt);
    }

    g_renderingEngine.SwapBuffers();
}
```

Per-Object Updates Are Inadequate

- Most low-level subsystems have stringent performance constraints
- They benefit from **batched updating**
- More efficient to update many animations in one batch
  - Rather than interleaved with collision, physics, rendering
- Most engines update each subsystem from the main loop, not per-object

Subsystem-Owned State

- A game object asks the subsystem to allocate state on its behalf
- Example: a renderable object asks for a mesh instance
- The mesh instance keeps position, orientation, scale, material data
- The rendering engine maintains a collection of mesh instances internally
- Object controls how it’s rendered by manipulating the mesh instance
- Object does not control rendering directly
- After all updates, the rendering engine draws all visible instances in one batch

A Better Tank Update

```
virtual void Tank::Update(float dt)
{
    // Update the state of the tank itself.
    MoveTank(dt);
    DeflectTurret(dt);
    FireIfNecessary();

    // Control the properties of my various engine subsystem components, but do NOT update them here...

    if (justExploded)
        m_pAnimationComponent->PlayAnimation("explode");

    if (isVisible)
    {
        m_pCollisionComponent->Activate();
        m_pRenderingComponent->Show();
    }
    else
    {
        m_pCollisionComponent->Deactivate();
        m_pRenderingComponent->Hide();
    }
}
```

A Better Game Loop

```
while (true)
{
    PollJoypad();

    float dt = g_gameClock.CalculateDeltaTime();

    for (each gameObject)
    {
        gameObject.Update(dt);
    }

    g_animationEngine.Update(dt);
    g_physicsEngine.Simulate(dt);
    g_collisionEngine.DetectAndResolveCollisions(dt);
    g_audioEngine.Update(dt);
    g_renderingEngine.RenderFrameAndSwapBuffers();
}
```

Benefits of Batched Updating

- Maximal cache coherency:
  - per-object data laid out contiguously in RAM
- Minimal duplication of computations:
  - global calculations done once
- Reduced reallocation of resources:
  - allocate once per frame
- Efficient pipelining:
  - scatter/gather across multiple cores
- Some subsystems simply don't work per-object because multi-body collision resolution must consider objects as a group

Object and Subsystem Interdependencies

- Even ignoring performance, per-object updates break down with dependencies
- Example: a human character holding a cat
  - Cat's skeleton pose depends on the human's pose
  - Update order matters
- Subsystems can also depend on one another
- Example: rag doll physics depends on animation
  - Animation produces local-space pose
  - Joints converted to world space, applied to rigid bodies
  - Physics simulates, joints written back, animation finalizes

Phased Updates

- Account for subsystem dependencies by ordering them in the loop

```
while (true) // main game loop
{
    // ...

    g_animationEngine.CalculateIntermediatePoses(dt);
    g_ragdollSystem.ApplySkeletonsToRagDolls();
    g_physicsEngine.Simulate(dt); // runs ragdolls too
    g_collisionEngine.DetectAndResolveCollisions(dt);
    g_ragdollSystem.ApplyRagDollsToSkeletons();
    g_animationEngine.FinalizePoseAndMatrixPalette();

    // ...
}
```

Multiple Update Hooks

- Game objects may need updates at multiple points during the frame
- Example: an object wants to procedurally adjust an intermediate animation pose
- Must update once before animation, once after
- Naughty Dog updates objects three times:
  - Once before animation blending
  - Once after animation blending but before final pose generation
  - Once after final pose generation
- Implemented via three virtual "hook" functions

Phased Update Loop

```
while (true) // main game loop
{
    // ...

    for (each gameObject)
        gameObject.PreAnimUpdate(dt);

    g_animationEngine.CalculateIntermediatePoses(dt);

    for (each gameObject)
        gameObject.PostAnimUpdate(dt);

    g_ragdollSystem.ApplySkeletonsToRagDolls();
    g_physicsEngine.Simulate(dt); // runs ragdolls too
    g_collisionEngine.DetectAndResolveCollisions(dt);
    g_ragdollSystem.ApplyRagDollsToSkeletons();
    g_animationEngine.FinalizePoseAndMatrixPalette();

    for (each gameObject)
        gameObject.FinalUpdate(dt);
```

Issues with Per-Object Hook Iteration

- Iterating all game objects for every hook is expensive
- Calling a virtual function on every object adds up
- Most objects don’t need every phase
- Iterating over uninterested objects wastes CPU
- Inflexible: only game objects supported (what about particle systems?)
- Better design: a generic callback mechanism
  - Subsystems provide registration for each phase
  - Any client (objects, particle systems, etc.) can register

Inter-Object Dependencies

- Objects may also depend on each other
- Example: a character holding a weapon needs the character's pose first
- Inter-object dependencies form a **forest of dependency trees**
  - Roots: objects with no dependencies
  - First-tier children: depend on root objects
  - Second-tier children: depend on first-tier, etc.

Inter-Object Dependencies

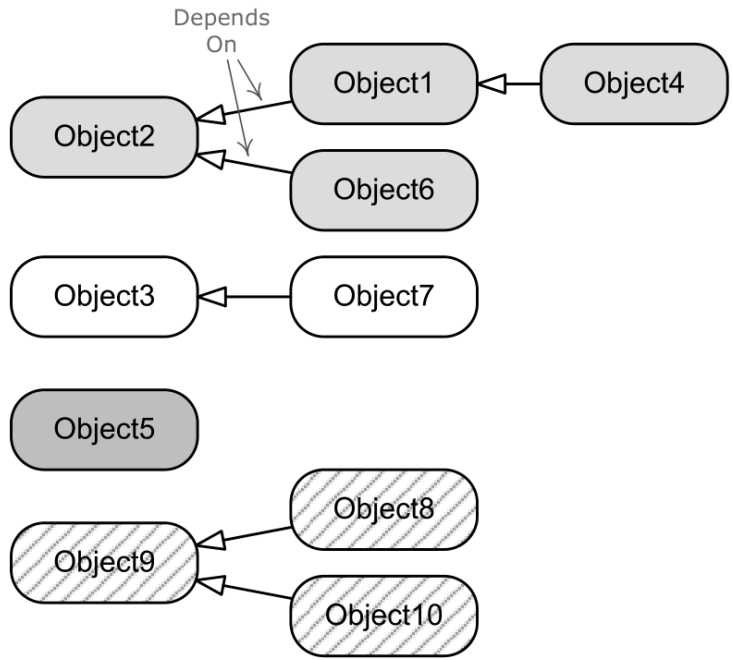


Figure 16.14. Inter-object update order dependencies can be viewed as a forest of dependency trees.

Bucketed Updates

- Collect objects into independent groups called **buckets**
- First bucket: all root objects
- Second bucket: all first-tier children
- Third bucket: all second-tier children, etc.
- For each bucket, run a complete update of objects and engine systems
- In practice, dependency trees are usually shallow
  - Example: vehicles, characters on vehicles, weapons in hands = three buckets
- Many engines limit the depth, using a fixed number of buckets

Bucketed Updates

```
enum Bucket
{
    kBucketVehiclesPlatforms,
    kBucketCharacters,
    kBucketAttachedObjects,
    kBucketCount
};
```

Bucketed Updates

```
void UpdateBucket(Bucket bucket)
{
    // ...

    for (each gameObject in bucket)
        gameObject.PreAnimUpdate(dt);

    g_animationEngine.CalculateIntermediatePoses
     (bucket, dt);

    for (each gameObject in bucket)
        gameObject.PostAnimUpdate(dt);

    g_ragdollSystem.ApplySkeletonsToRagDolls(bucket);
    g_physicsEngine.Simulate(bucket, dt); // ragdolls etc.
    g_collisionEngine.DetectAndResolveCollisions(bucket, dt);
    g_ragdollSystem.ApplyRagDollsToSkeletons(bucket);
    g_animationEngine.FinalizePoseAndMatrixPalette(bucket);

    for (each gameObject in bucket)
        gameObject.FinalUpdate(dt);

    // ...
}
```

Game Loop with Buckets

```
void RunGameLoop()
{
    while (true)
    {
        // ...

        UpdateBucket(kBucketVehiclesAndPlatforms);
        UpdateBucket(kBucketCharacters);
        UpdateBucket(kBucketAttachedObjects);

        // ...

        g_renderingEngine.RenderSceneAndSwapBuffers();
    }
}
```

This approach is used at Naughty Dog for all *Uncharted* games and *The Last of Us.*

State Inconsistencies

- In theory, all object states are updated instantaneously and in parallel from t_1 to t_2
- In practice, single-threaded loops update objects one at a time
- Halt halfway: half of objects are at $S_i(t_2)$, half are still at $S_i(t_1)$
- Two objects may disagree on what time it is!
- Some objects may be only partially updated
- States are consistent before and after the update loop, but may be inconsistent during it

One-Frame-Off Lags

- Inconsistency causes confusion and bugs
- Object B looks at object A's velocity to compute its own velocity at t
- Programmer must be clear: do they want $S_A(t_1)$ or $S_A(t_2)$?
- If $S_A(t_2)$ is needed but A hasn't updated, we get an **update order problem**
- This causes **one-frame-off lags**
- Manifests as lack of synchronization between objects on screen

Object State Caching

- Bucketing helps but imposes arbitrary query limitations
  - An object should never query an object in its own bucket safely
- State caching: each object caches $S_i(t_1)$ while computing $S_i(t_2)$
- Benefits:
  - Any object can query previous state without regard to update order
  - Consistent state always available, even during updates
- Bonus: linearly interpolate previous and next states
  - Havok physics maintains previous and current states for this reason
- Downside: doubles memory; only solves half the problem

Time-Stamping

- Low-cost way to improve consistency: time-stamp object states
- Easy to determine if a state corresponds to previous or current time
- Code that queries another object can assert or check the time stamp
- Set a global to track which bucket is currently updating
- Each object knows its bucket; check that the queried object is in a different bucket

High-Level Game Flow

Controlling Game Flow

- The object model defines what objects exist and how they behave
- Says nothing about player objectives, success, or failure
- Need a system to control **high-level game flow**
- Often implemented as a **finite state machine**
- Each state usually represents a single objective or encounter
- Associated with a particular locale in the game world

The High-Level State Machine

- As the player completes a task, the FSM advances to the next state
- Player presented with a new set of goals
- FSM defines what happens on failure
  - Often: return to the start of the current state
- After enough failures, player runs out of "lives"
  - Sent back to the main menu
- Flow of the entire game (menus, first level, last level) controlled by the FSM

Key Takeaways

Key Takeaways

- The runtime object model can be:
  - object-centric (class instances)
  - property-centric (data tables keyed by id)
- Monolithic class hierarchies suffer from deep/wide problems and the bubble-up effect
- Favor composition over inheritance to simplify hierarchies
- Component-based designs decouple game object features
- Property-centric designs are more memory-efficient and cache-friendly

Key Takeaways

- **Batched updates** beat per-object subsystem updates for performance
- **Phased and bucketed updates** address inter-object and inter-subsystem dependencies
- **State caching** and **time-stamping** mitigate inconsistencies during updates
- Game flow is typically controlled by a **finite state machine**

Participation Exercise

No participation exercise for today.

Please complete the SRT!

The last participation exercise will be **in-person** in the class on Monday.

Questions?